Detecting Hidden ML Training With Zero-Overhead Telemetry

Robi Rahman[*] **Sabiha Tajdari**[†]

Abstract

Hardware-enabled monitoring of GPU workloads underpins many proposals for AI compute governance, but if developers can defeat monitoring mechanisms, such schemes are unworkable. We evaluate the adversarial robustness of GPU workload classification using only zero-overhead NVML telemetry: content-agnostic signals that observe physical effects of computation without slowing execution or accessing model weights, training data, or hyperparameters. Across 6 rounds of monitor-evader iteration, we evaluate 22 evasion strategy families on 9 GPU models spanning 4 architecture generations. We develop a classifier that achieves 98.7% binary accuracy at identifying training workloads across the whole corpus, and 12–91% accuracy against the most challenging unexpected workloads even when they are adversarially disguised.

1 Introduction

Compute governance proposals such as [Shavit, 2023, Baker et al., 2025, Scher et al., 2025] call for regulations including evaluations, audits, and registration or prohibition of training runs above specified compute thresholds [Heim and Koessler, 2024]. US Executive Order 14110, in effect from October 2023 until rescinded in January 2025, previously required developers which perform training runs exceeding 10^{26} floating-point operations (FLOP) to report those runs to the federal government, along with their cybersecurity and risk management practices [The White House, 2023].

However, the ability to regulate ML training compute requires that regulators are able to know when training occurs, and that developers cannot discreetly run training workloads undetected. Work is underway to establish the feasibility of geolocation of computing hardware [Brass and Aarne, 2025] and registries of AI chips [Fist and Grunewald, 2023]. But registration and geolocation are insufficient unless the regulator understands what the chips are doing.

Past work demonstrates the feasibility of distinguishing ML from non-ML computing when the adversary is not trying to disguise the workloads [Tang et al., 2022, Weakley et al., 2025], and existing governance literature discusses masking or structuring as possible evasion tactics [Heim et al., 2024, Seferis and Fist, 2025]. However, there has not yet been a systematic effort to iterate on this cat-and-mouse dynamic and determine whether the problem landscape favors detection or evasion.

Therefore, we investigate the adversarial robustness of GPU workload classifiers, trained on signals extracted from built-in telemetry readings. We build the classifier as a prerequisite, then extensively red-team it with 20+ categories of real and adversarial workloads across 100+ GPU-hours of ML experiments.

We find that zero-overhead GPU telemetry is sufficient to classify workloads as training vs non-training with 98.7% accuracy across 9 GPU types spanning consumer and datacenter hardware

[*]Machine Intelligence Research Institute. Correspondence to robi@intelligence.org.
[†]University of Virginia.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

(Ampere through Blackwell). Crucially, this telemetry is content-agnostic: it observes physical effects of computation (power, bandwidth, utilization) without accessing model weights, training data, or hyperparameters [Shavit, 2023, O'Gara et al., 2025].

Through 6 rounds of monitor-evader iteration, we implement 22 families of evasion strategies, including 4 multi-GPU model- and/or data-parallel configurations on nodes of up to 8 datacenter cards. Leave-one-strategy-out evaluation — training classifiers on some evasion tactics while evaluating on unknown workloads — shows mean detection rate of 85.3% across 22 unseen strategies, with all multi-GPU workloads detected at 100% and only 6 of 22 families under 90% detection against the best classifier. Physics-based evasion-resistant features (cumulative energy, power-memory correlation, pre-allocation memory-plateau features) provide the signal needed to catch the hardest remaining single-GPU strategies.

We release a content-agnostic classifier achieving 98.7% accuracy on training, inference, and non-ML workloads run on NVIDIA hardware in generations Ampere through Blackwell, using only zero-overhead NVIDIA Management Library (NVML) telemetry signals. Source code for the non-ML, inference and training, and adversarial evasion workloads is available in the project repository. We invite the AI governance and ML research communities to red-team our classifier and publish new evasion strategies that might defeat it, which will make it even more robust in the future.

2 Background and Threat Model

2.1 Telemetry Signals

Our classifier operates on nine always-available NVML counters (GPU utilization, memory utilization, memory used, power draw, temperature, SM clock, memory clock, PCIe TX/RX) sampled at 1 Hz. These counters are available on every NVIDIA GPU from Pascal onwards with no runtime overhead: polling them does not measurably burden or slow down the monitored workload, and does not require any additional configuration. The readings do not involve access to application code, model weights, or training data — these signals are purely physical or electrical measurements of utilization of different components of the GPU.

By contrast, existing profiling-based monitoring proposals carry prohibitive costs. WAVE [Xu et al., 2026] achieves privacy-preserving oversight using Nsight Compute to read performance counters, but this adds 1200–5300% runtime overhead: Nsight Compute is a kernel-debugging profiler not designed for production monitoring, and is very inefficient to run continuously along with workloads. NVML counters are a more efficient choice, and more sustainable since datacenters are already using huge and rapidly increasing amounts of energy [Chen et al., 2025a, You and Owen, 2025]. Our result establishes a lower bound on the efficacy of convenient, economical oversight: anything that works with this minimal observational budget also works with more information, so a real deployment monitor that can access more telemetry or physical channels starts from this demonstrated detection level and only becomes stronger.

2.2 Threat Model

We consider two types of evaders. One is a cloud tenant: a user or AI developer renting GPU time, attempting to disguise ML training as inference or non-ML workloads to evade the cloud provider's monitoring. Proposals such as Heim et al. [2024] rely on the intermediary role of compute providers acting as "record keepers, verifiers, and, in some cases, even enforcers" of AI governance. In this case, the adversary controls user-space software but not the host OS, hypervisor, or hardware. One type of evasion they might attempt is *compute structuring*, wherein training runs are split across multiple cloud providers to dodge a reporting threshold. Seferis and Fist [2025] argue that these attacks are detectable, but this relies on the ability of each cloud host to reliably classify the workloads running on its own GPUs. If tenants can disguise training as other workloads and evade recognition by the host, the compute-structuring defense collapses. Our classifier provides the foundation that allows this type of enforcement to work, and our adversarial evaluation is a test of its robustness.

Another type of evader is a hardware operator. They own or operate the hardware, typically in a datacenter facility, and attempt to hide unauthorized AI training from an external regulator. The adversary controls the full software stack and can modify workload scheduling. In order to prevent this evader from defeating the monitoring scheme, we need to equip their chips with a *hardware-*

enabled governance mechanism (HEM) that ensures that the evader cannot (i) disable NVML polling or tamper with the returned counter values at the hardware level, nor (ii) inject synthetic telemetry into the monitor's ingestion channel. Both assumptions describe mechanisms the software stack alone cannot enforce; they are the minimum hardware-enabled-mechanism (HEM) guarantees our results depend on [Aarne et al., 2025, O'Gara et al., 2025]. Appendix J discusses the HEM requirements in more detail as a limitation and as a direction for development.

The hardware operator is theoretically more challenging to monitor than the cloud tenant, but both evaders are subject to a fundamental physical constraint: a backward pass requires ~2× the FLOPs of a forward pass, and therefore flips more transistors and consumes more energy. Evasion strategies designed against one category of evader transfer reasonably well to the other, because one major constraint is the physical energy requirement, not specific software.

2.3 Classification Task

Our dataset contains telemetry records from our 93 distinct ML training workloads (34 ordinary ML tasks, 59 adversarial workloads disguised as inference or non-ML), 40 distinct ML inference workloads, and 19 non-ML workloads. Our classifier can distinguish these with high accuracy and precision, as described in Methodology and Results below. But the fundamental governance-relevant question is binary: **Is a monitored GPU being used for training?** We tested several options before selecting a gradient-boosted trees classifier, which exhibited the best performance on this task during cross-validation. Details are provided in Appendix B.

3 Methodology

3.1 Data Collection

3.1.1 Hardware

Hardware used for this project includes 9 models of NVIDIA GPUs across 4 architecture generations. Consumer hardware is generally not covered by proposed AI governance and verification [Heim et al., 2024, Scher et al., 2025], but was included to test generalization across different hardware.

GPU Model[3]	Architecture	VRAM	Context	GPUs
RTX 4050 Laptop	Ada Lovelace	6 GB	Local laptop	1
RTX 5080	Blackwell	16 GB	Local desktop	1
RTX 5090	Blackwell	32 GB	Cloud	1, 2
A100 SXM4 40GB	Ampere	40 GB	Cloud	1, 8
A100 SXM4 80GB	Ampere	80 GB	Cloud	1, 8
H100 80GB	Hopper	80 GB	Cloud	1, 8
H100 NVL	Hopper	94 GB	Cloud	1, 2
H200 / H200 NVL	Hopper	141 GB	Cloud	1, 2, 8
B200	Blackwell	192 GB	Cloud	1, 2, 8

Table 1: Instances used in this research by type and quantity of GPUs. Some experiments involved fewer than the maximum number of GPUs available on the instance.

3.1.2 Workloads

We conducted 1,566 runs of 194 distinct workloads, totaling ~164 GPU-hours (~87 wall-clock hours) of experiments, for a total of 601,112 telemetry readings × 9 signals per reading. Not every workload is run on every hardware configuration, for instance due to memory limitations.

We conducted 956 runs of ML training, including 180 of supervised training (e.g. ResNet [He et al., 2016] on CIFAR-10, GPT-2 on WikiText, BERT [Devlin et al., 2019] fine-tuning), 72 of distributed data-parallel training, 152 of fully-sharded data-parallel training on models with 7B to 70B parameters, and 42 of LoRA fine-tuning. This also included 481 runs of adversarial evasion, testing strategies such as utilization modulation, memory minimization or preallocation, clock throttling, interleaving training with inference or non-ML workloads, mimicking FFT computations or cryptocurrency mining, and padding with idle time.

Category	Workloads	Runs
ML training	125	956
ML inference	50	419
Other	19	191

Table 2: Number of workloads and runs in each of the three overall categories.

For inference, we did 419 runs, including 49 of image classification, 264 of LLM inference (TinyLlama, Phi-3, Qwen-2.5 3B through 72B, Mistral and Mixtral, DeepSeek-V3 and R1), and 39 of other modalities (e.g. Whisper speech-to-text, SDXL text-to-image diffusion).

We also did 191 runs of non-ML workloads, including 65 of scientific HPC (e.g. fast Fourier transform, n-body simulation), 36 of cryptocurrency mining, 36 of rendering or video processing (Blender, rendering, FFMPEG), 34 idle-GPU baselines, and 14 miscellaneous non-ML runs.

A full catalog of adversarial workloads is provided in Appendix B.4 and source code for every workload is available in the project repository.

3.1.3 Telemetry

During all workloads, we sample 9 raw NVML signals at 1 Hz:

- GPU utilization (%)
- Memory utilization (%)
- Memory used (MB)
- Power draw (W)
- Temperature (°C)
- SM clock (MHz)
- Memory clock (MHz)
- PCIe TX bandwidth (MB/s)
- PCIe RX bandwidth (MB/s)

Additionally, for 49 workloads tested on the local RTX 5080 desktop, we validate the telemetry's reported power draw using an external Tapo P115 smart plug reading the system's power draw with timestamps matching within 0.5s.

We extract features from the raw signals as described in Appendix B.3.

3.2 Classifier

We selected the best-performing of several candidate models. XGBoost [Chen and Guestrin, 2016] was the most accurate, and RandomForest [Breiman, 2001] was a close runner-up. We use XGBoost for all reported headline results. SVM-RBF and logistic regression were also considered, but exhibited worse performance during cross-validation. Classifier comparison is provided in Appendix B.2.

The feature set is designed along two signal layers: (1) a structural, roofline [Williams et al., 2009] layer that identifies workloads occupying a training-like operating region in arithmetic-intensity space (mean power, mean utilization), and (2) a temporal telemetry layer that resolves ambiguity within that region using time-domain features (autocorrelation, CV, periodicity). Both stages of the two-stage pipeline use the same underlying feature vocabulary: stage 1 retains the full set; stage 2 drops the level features to reduce overfitting and prevent exploitation by adversarial strategies that mimic specific usage levels.

3.3 Adversarial Evaluation Protocol

Our refinement procedure iterates on the classifier and the evader through six rounds of co-evolution. In each round, we design new evasion strategies for the attacker, targeting the classifier's existing

weaknesses, and then the defender trains a hardened classifier incorporating the new adversarial examples. Qualitatively, the classifier improves through stages which can be described as *baseline* — trained only on non-adversarial data consisting of real-life workloads — to *hardened* — defeating initial adversarial strategies introduced in earlier rounds — to *evasion-resistant* — hardened against more sophisticated obfuscation, plus sensing physics-based features that capture fundamental computational invariants that the adversary cannot suppress without aborting training.

Round	Attacker introduces	Defender response
1	Utilization manipulation: modulation (A), low-utilization training (B), temporal disruptions (D)	Retrain → **hardened** classifier
2	Memory reduction and non-ML mimicry: minimal-memory training (E), interleaving (F), clock throttling (G), FFT/mining mimicry (H), stochastic (I), PID control (J)	Retrain → **hardened** (improved)
3	Training-as-inference: online learning (K), training diluted with N $\in$ {2,5,10,20} $\times$ inference (L)	Add power-memory correlation + cumulative energy features → **evasion-resistant**
4	Composite evasion: min-memory composite strategy (M), gradient accumulation structuring (N), composite idle-padding sweep (0-95% idle fraction); multi-GPU distributed data-parallel variants (K_ddp, K_ddp_accum, L_ddp, L_ddp_stagger)	Retrain on multi-GPU data → **evasion-resistant** (improved)
5	White-box (attacker given classifier internals): diluted full-model fine-tuning (WB-D), diluted LoRA fine-tuning (WB-L), WB + grad-accum combined	Add pre-allocation memory-plateau features (first_30s_mem_delta, time_to_mem_plateau); collect more non-adversarial examples of matched-size dense LLM inference on datacenter hardware
6	Symmetric-GEMM backprop disguise: fuse SGD weight update into dW epilogue so NVML cannot see optimizer state writes	Retrain on symmetric-GEMM inference controls, distributed RL, and PPO-RLHF

Table 3: Summary of adversarial iteration over several rounds of increasing evader and classifier sophistication. Workload labels are in parentheses; a full catalog is provided in Appendix B.4.

This iteration process yielded 22 families of evasion strategies covering six technique classes: utilization manipulation, memory reduction, workload mimicry, training-as-inference (e.g. dilution or mixing the two), multi-GPU evasion (e.g. structuring workloads across multiple GPUs to look less like training on some of them individually), white-box adversary (examining previous classifiers to discover niche weaknesses), kernel-level and tensor operation disguises, and composite strategies combining more than one of these techniques.

For each strategy family, we train the classifier on real (non-adversarial) training, inference, and non-ML workload data, plus every *other* adversarial strategy, and test on the held-out family. This tests the classifier's ability to generalize to unseen, novel evasion techniques, which informs us about the real-world governance case where an adversary invents a strategy we have not yet catalogued.

4 Results

4.1 Baseline Classification

First we present the non-adversarial, pre-hardening baseline. We replicate the previous finding that the classification problem is solvable and real training and inference workloads can be distinguished from non-ML workloads [Tang et al., 2022, Xu et al., 2026] such as cryptocurrency mining and graphics rendering.

Pooling all 9 GPU models, the classifier achieves 98.70% $\pm$ 1.23% accuracy with macro-F1=0.986.[4] Predicting classes of held-out runs, it achieves 99.24% precision and 98.31% recall, with 98.99% specificity and 1.01% false-positive rate. Latency is quite quick, with 98.02% accuracy at 5s windows and 98.70% at 30s. Analysis of detection latency is provided in Appendix E. Because the classifier

[4]XGBoost (400 trees, max_depth 6, learning rate 0.1, histogram tree method), 30 s windows, stride 15 s, 5-fold grouped cross-validation holding out whole jobs, 36,666 windows across 1,558 streams in 1,198 jobs that produced at least 5 NVML samples.

emits a calibrated P(training), the decision threshold can be adjusted depending on the context, without retraining. For example, a regulator monitoring a facility for covert training would prefer high sensitivity, catching every training run even at the cost of more false positives, whereas a compute provider would likely prefer to use the classifier with a higher training threshold to avoid flagging legitimate customers' inference workloads.

Accuracy was evaluated with the data split by GPU model, comparing single-GPU training against training on data pooled across all 9 GPUs:

GPU	Single-GPU CV	LOGO
A100-SXM4 40GB	99.91%	99.12%
A100-SXM4 80GB	99.96%	98.05%
B200	98.54%	91.43%
H100 NVL	91.64%	87.37%
H200	98.68%	86.40%
H100 80GB HBM3	98.97%	97.97%
RTX 5090	96.09%	80.48%
RTX 5080	98.01%	77.59%
RTX 4050 Laptop	94.74%	66.75%

Table 4: Classification accuracy by GPU. The Single-GPU CV column trains and tests on different workloads from the same GPU using 5-fold grouped cross-validation (Runs are randomly allocated to a single fold; all windows from every GPU's telemetry from the same run are within the same fold, so data from a run never appears in both train and test sets). The LOGO (leave-one-GPU-out) column trains on data from the *other* 8 GPUs and tests on the held-out GPU, which is the real-world scenario when the classifier is deployed to new hardware not in the training set. In-distribution pooled CV, training on workloads from *all* GPUs, achieves 98.70% $\pm$ 1.23% on unseen workloads across all GPUs.

In-distribution pooled classification is highly accurate for non-adversarial workloads: when data from all 9 GPUs is used to develop the classifier, it is 98.70% accurate on new, non-disguised workloads performed on any of those GPUs. A harder case is when the classifier is developed using workload logs on a set of GPUs and then it is deployed to classify telemetry from a novel GPU. This case is evaluated in the LOGO column of Table 4. Our results show that a regulator with access to telemetry from a variety of GPUs can train one classifier and deploy it even to datacenter GPU models they have not sampled before at 86–99% accuracy. Generalization to consumer hardware is weaker, at 67–80%, so a deployment that must cover consumer GPUs should be trained on more telemetry from that category.

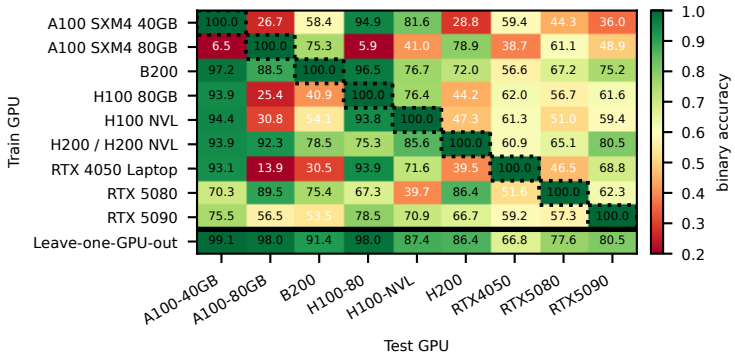


Figure 1: Cross-GPU transfer heatmap (binary XGB, 30s windows). Cell values are accuracy when training on the row GPU and testing on the column GPU. The bottom row is LOGO: for each test GPU, the classifier is trained only on data from the other 8 GPUs. By contrast, *in-distribution* evaluation (with runs from the target GPU included in the training dataset) averages 98.70% accuracy on unseen non-adversarial runs. LOGO evaluates the harder case of generalization to new hardware types.

4.2 Adversary-Defender Iteration

The arms race proceeds through six rounds. Table 5 summarizes the classifier's detection rates and the adversary's evasion strategies.

Round	Attacker capability	Baseline classifier detection of new evader	Previous round classifier detection of new evader	Hardened detection in-distribution	Hardened detection, unseen workloads
Baseline	(No evasion; real workloads only)	N/A			**98.4%**[5]
1	Utilization manipulation (A, B, D)	30–86%[6]		99–100%	**30–93%**
2	Memory reduction / mimicry (E, F, G, H, I, J)	15–77%	36–97%	96–100%	**92–100%**
3	Training-as-inference (K online, L diluted)	89–99%	87–100%	90–100%	**87–100%**
4 (Single-GPU)	Composite + grad accum (M, N, composite 0–95 % idle)	10–100%	82–100%	100%	**85–100%**
4 (Multi-GPU)	Multi-GPU distributed evasion strategies (K/L DDP + stagger)	98–100%	99-100%	100%	**100%**
5	White-box (WB-full, WB-LoRA) with R4 classifier internals exposed	50–97%	52–90%	92–100%	**56–91%**
6	Matmul disguise: backprop reshaped to appear as forward passes (fused_sgd); RL training with updates diluted across PPO/RLHF loop (ppo_fused)	12–100%	9–99%	87–100%	**12–96%**

Table 5: Arms race summary showing each classifier state's detection rate against each round's evasion strategies. Detection rate for each strategy is computed over runs that employ that strategy; workloads not affected by the strategy are not included. Baseline classifiers are trained only on non-adversarial data. In each round, the adversary introduces new strategies to defeat the previous version of the classifier, then the defender teaches the classifier to recognize training disguised with the new evasion strategies.

The first row shows baseline classification performance on only clean, non-adversarial workloads. The baseline classifier, trained on all but one job at a time, achieves an average of 98.4% accuracy on the other, held-out job.

In Round 1, the evader introduces workload obfuscation strategies revolving around utilization manipulation. The baseline classifier achieves 30–86% binary accuracy at classifying workloads using these strategies as training vs not-training. Then the defender iterates on these strategies, achieving 99–100% in-distribution accuracy at classifying these workloads after it has seen them, but only 30–93% accuracy on strategies from Round 1 it has not seen.

In Round 2, the evader introduces memory reduction and mimicry strategies, and in Round 3, the evader introduces so-called "training-as-inference" strategies, camouflaging online learning as inference or diluting gradient updates between many batches of inference. Each time, the evader's new strategies achieve partial success against the baseline classifier and the previous round's classifier, and each time, the defender trains the new classifier to reliably detect the new evasive workload if it has seen that strategy, or detect them the wide majority of the time if it has seen strategies from that round but not the specific strategy it is tested on.

In Round 4, we introduced new evasion strategies for disguising single-GPU training, as well as strategies for disguising multi-GPU training which attempt to prevent the detection of training on each GPU's telemetry by utilizing other GPUs to shift the workload around. Round 4's strategies achieved some initial evasion against the classifier from Round 3, but were very thoroughly rendered ineffective once the classifier was hardened against them.

In Round 5, we gave the evader white-box access to the classifier developed in previous rounds. The evader was able to examine the internal mechanics of the classifier to find precise exploits. For example, while red-teaming the Round 4 classifier, we noted that the classifier partly relies on features derived from windows 5s or longer. Therefore, one prospective evader strategy is to insert 4s of training across the boundary of two 5s windows, spending 40% of 10s on training but taking up less than half of the time of each window, misleading the classifier towards identifying this as the other workload that was running. Another tactic was to use a copy of the Round 4 classifier against

the defender in real-time, running training as long as the defender believed P(training)<0.30 and temporarily replacing training with another workload whenever the defender became suspicious. These tactics achieved up to 48% evasion against the Round 4 classifier. However, once the defender was able to train against them, they were rendered ineffective, with 92–100% detection of seen evasion strategies, and notably 56–91% detection of unseen white-box strategies after hardening against other Round 5 strategies.

Round 5 represents a disaster scenario for governance, equivalent to a developer infiltrating the governing agency, stealing the classifier, and orchestrating their workload in a way that is designed to be maximally undetectable by the classifier's exact algorithm.[7] However, once a classifier is hardened against a few such exploits, other white-box attacks are much less effective in the future.

Finally, in Round 6, we experimented with custom kernels and matrix operations rather than using standard PyTorch implementations. We implemented a kernel that makes the computational signature of backpropagation identical to that of two forward passes, then replicated the effect using a custom torch function. The only essential difference that could not be eliminated was that training writes weights, so the disguise folds the weight update into the multiplication that produces it, keeping no gradient and no optimizer state. This is the most challenging evasion in the corpus: the baseline classifier catches it only 12% of the time, and hardening reaches only 87% even in-distribution, the weakest result in the paper.

5 Discussion

5.1 An Involuntary Witness

During ML training, a backward pass (computing gradients at each parameter) involves twice as much computation as the forward pass [Kaplan et al., 2020]. This imposes a fundamental constraint on training: for the same number of tokens processed, it is three times as energy-intensive as inference, so you can only either shuffle energy consumption around or reduce the amount of training conducted, but ultimately you cannot make them use the same amount of energy without eliminating the step of updating the weights, which would leave the workload unable to train the model.

Our empirical evidence is consistent with this and suggests that power draw is a valuable feature for distinguishing training from inference.[8] Speculatively, as we iterate further on the classifier-evader process in the future, evaders may learn to defeat other features, but energy is a fundamental constraint, so energy-based features may rise in importance as the classifier hardens further.

5.2 Single- vs Multi-GPU Detection

Our datacenter experiments (on 8-GPU A100, H200, B200 nodes) test both threat models above. We designed 4 multi-GPU evasion strategies: DDP online learning, DDP diluted training, staggered dilution (spreading NVLink traffic across time), and gradient accumulation across ranks. All are detected at 99.6–100% in leave-one-strategy-out evaluation (Table 7; Figure 5). Multi-GPU training is harder to hide than the much less important single-GPU case because traffic and interconnect usage patterns that differ between ordinary multi-GPU large-model training and inference provide additional opportunities for the classifier to detect differences in signals, and more that the evader will have to convincingly disguise. For example, our classifier learned to recognize AllGather and ReduceScatter NVLink traffic associated with forward/backward steps in FSDP training at 7B-70B scale and applied this to detect obfuscated multi-GPU training even when the adversary tried to manipulate the communication patterns. Multi-GPU training may create distinctive telemetry signatures on each GPU even when workloads are structured so that an individual GPU shows only moderate utilization.

[7] In an absolute worst-case scenario, the developer could infiltrate the regulator, replace the classifier with a compromised version containing a backdoor, and also compromise the monitored hardware.

[8] This theoretical result is corroborated by our experiments as well as evidence from large-scale training and inference deployments. For example, POLCA [Patel et al., 2023] find that inference uses less power than training in Microsoft Azure cloud datacenters.

5.3 Privacy Preservation

The NVML metrics queried by our classifier are content-agnostic: it observes the physical and architectural effects of computation (power draw, temperature, utilization, memory) without accessing model weights, training data, hyperparameters, or application logic. This addresses a central deployment concern in shared cloud settings, where governance mechanisms that require inspecting proprietary information face legal, operational, and ethical objections [Shavit, 2023, O'Gara et al., 2025]. Our classifier demonstrates that the governance-relevant question ("is this GPU training a model?") can be answered without needing access to the semantic content being processed. Notably, side channels such as power, thermal, and timing traces can still leak clues about the workload content [Tang et al., 2022, Chaudhuri et al., 2025]; further research should investigate the minimum set of data that can be collected by the monitor while still robustly classifying the workload category.

5.4 Layered Defense

If telemetry is collected and monitored at multiple layers (device, cluster, facility), an adversary must evade all layers simultaneously [O'Gara et al., 2025]. The evasion cost is likely cumulative across layers, in which case our NVML-only results understate what a layered monitor can achieve. DCGM counters (tensor-core activity, NVLink throughput) and wall- or rack-level power measurements provide additional signal. If combined with tamper-resistant hardware such as flexHEGs [Petrie et al., 2025], software-level evasion alone would face an even harder task.

6 Conclusion

Zero-overhead GPU telemetry monitoring is a viable mechanism for compute governance. Defenders are structurally favored, especially in the most relevant case: our classifier is extremely effective at detecting all evasion strategies tested on multi-GPU datacenter nodes, correctly classifying 98-100% of unforeseen workloads in that setting. Single-GPU evasion has more success, with a mean evasion rate of 18.0% in held-out evaluation across 18 single-GPU strategies. Three families fall below 50% detection: backprop with its matrix operations disguised as forward passes (11.7%), and two utilization-manipulation strategies (30.0% and 34.2%). White-box LoRA, the hardest case in earlier rounds, is now detected at 56.5% even when evaders have access to the defender's previous classifier. The classifier learns to recognize fundamental features that cannot be eliminated without sacrificing the ability to update model weights.

The compute-monitoring proposals of Shavit [2023], Baker et al. [2025], and O'Gara et al. [2025] are empirically realisable against a realistic software-level evasion adversary, conditional on the HEM assumptions of tamper-resistant telemetry, an authenticated telemetry channel, and a secure classifier. With those hardware guarantees in place, the adversary is left with a physical cost trade-off (dilution and idle-padding) whose economics are unviable at frontier scale. By demonstrating that it is possible to construct a classifier that accurately identifies training activity, we narrow the open governance questions and the ML problem toward the hardware security problem: can we build the matching HEM, and close any gaps left by telemetry- and hardware-based controls? We invite researchers and ML experts to red-team our classifier and publish evasion strategies that might defeat it, so that we can iterate on any such findings to ultimately strengthen its robustness.

References

Onni Aarne, Tim Fist, and Caleb Withers. Secure, governable chips: Using on-chip mechanisms to manage national security risks from AI and advanced computing. Technical report, Center for a New American Security, 2025. URL https://www.cnas.org/publications/reports/secure-governable-chips.

Ghadeer Almusaddar, Yicheng Zhang, Sankha Ganjisaffar, Brandon Williams, Yu David Liu, Dmitry Ponomarev, and Nael Abu-Ghazaleh. ShadowScope: Composable performance-counter monitoring for GPU computation integrity. *arXiv preprint arXiv:2509.00300*, 2025. URL https://arxiv.org/abs/2509.00300.

Anonymous. Differential architecture: Hardware mechanisms to limit the performance of targeted applications. In *Proceedings of the 53rd International Symposium on Computer Architecture (ISCA)*, 2026. Forthcoming.

Sheikh Enamul Arefin and Abdul Serwadda. Exploiting HDMI and USB ports for GPU side-channel insights. In *Proceedings of the 15th ACM Conference on Data and Application Security and Privacy (CODASPY)*, 2025. arXiv:2410.02539.

Mauricio Baker, Gabriel Kulp, Oliver Marks, Miles Brundage, and Lennart Heim. Verifying international agreements on AI: Six layers of verification for rules on large-scale AI development and deployment. *arXiv preprint arXiv:2507.15916*, 2025. URL https://arxiv.org/abs/2507.15916.

Asher Brass and Onni Aarne. Location verification for AI chips. Technical report, Institute for AI Policy and Strategy, 2025. URL https://www.iaps.ai/research/location-verification-for-ai-chips.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

Arunava Chaudhuri, Shubhi Shukla, Sarani Bhattacharya, and Debdeep Mukhopadhyay. Energon: Unveiling transformers from GPU power and thermal side-channels. *arXiv preprint arXiv:2508.01768*, 2025. URL https://arxiv.org/abs/2508.01768.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016.

Xinyu Chen, Xi Wang, Aldo Colacelli, Minwoo Lee, and Le Xie. Electricity demand and grid impacts of AI data centers: Challenges and prospects. *arXiv preprint arXiv:2509.07218*, 2025a. URL https://arxiv.org/abs/2509.07218.

Zhuoran Chen, Steven W. D. Chien, Peng Qian, and Noa Zilberman. Hardware-centric, workload-agnostic anomaly detection for GPU computing. *arXiv preprint arXiv:2510.26008*, 2025b. URL https://arxiv.org/abs/2510.26008.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186, 2019.

Epoch AI. Data on AI chip sales, 2025. URL https://epoch.ai/data/ai-chip-sales.

Giuseppe Esposito, Juan-David Guerrero-Balaguera, Josie Esteban Rodriguez Condia, Matteo Sonza Reorda, Marco Barbiero, and Rossella Fortuna. Estimating GPU stress and reliability across workloads from telemetry and performance counters. *arXiv preprint arXiv:2511.05067*, 2025. URL https://arxiv.org/abs/2511.05067.

Tim Fist and Erich Grunewald. Preventing AI chip smuggling to china: A working paper. Technical report, Center for a New American Security, 2023. URL https://www.cnas.org/publications/reports/preventing-ai-chip-smuggling-to-china.

Ankit Gangwal, Samuele Giuliano Piazzetta, Gianluca Lain, and Mauro Conti. Detecting covert cryptomining using HPC. *arXiv preprint arXiv:1909.00268*, 2019. URL https://arxiv.org/abs/1909.00268.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

Lennart Heim and Leonie Koessler. Training compute thresholds: Features and functions in AI regulation. *arXiv preprint arXiv:2405.10799*, 2024. URL https://arxiv.org/abs/2405.10799.

Lennart Heim, Tim Fist, Janet Egan, Sihao Huang, Stephen Zekany, Robert Trager, Michael A. Osborne, and Noa Zilberman. Governing through the cloud: The intermediary role of compute providers in AI regulation. *arXiv preprint arXiv:2403.08501*, 2024. URL https://arxiv.org/abs/2403.08501.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL https://arxiv.org/abs/2001.08361.

Gabriel Kulp, Daniel Siu, and Lennart Heim. Hardware-enabled governance mechanisms: Developing technical solutions to exempt items otherwise classified under export control classification numbers 3a090 and 4a090. Technical report, RAND Corporation, 2025. URL https://www.rand.org/pubs/working_papers/WRA3056-1.html.

Imran Latif, Alex C. Newkirk, Matthew R. Carbone, Arslan Munir, Yuewei Lin, Jonathan Koomey, Xi Yu, and Zhiuha Dong. Empirical power signatures of large-scale training on 8-GPU H100 nodes. Technical report, Office of Scientific and Technical Information (OSTI), 2025. URL https://www.osti.gov/biblio/2555926.

Henrique Teles Maia, Chang Xiao, Dingzeyu Li, Eitan Grinspun, and Changxi Zheng. Can one hear the shape of a neural network? snooping the GPU via magnetic side channel. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4383–4400. USENIX Association, 2022.

Sajjad Kiani Monfared, Fatemeh Ganji, Daniel Holcomb, and Shahin Tajik. Detecting ML training under untrusted host and device via timing and memory observables. *arXiv preprint arXiv:2602.09369*, 2026. URL https://arxiv.org/abs/2602.09369.

Aidan O'Gara, Gabriel Kulp, William Hodgkins, James Petrie, Vincent Immler, Aydin Aysu, Kanad Basu, Shivam Bhasin, Stjepan Picek, and Ankur Srivastava. Hardware-enabled mechanisms for verifying responsible AI development. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2505.03742.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. POLCA: Power oversubscription in LLM cloud providers. *arXiv preprint arXiv:2308.12908*, 2023. URL https://arxiv.org/abs/2308.12908.

James Petrie. Offline licensing: A proposal for authorizing AI chip operation. Technical report, Institute for AI Policy and Strategy, 2025. Working paper.

James Petrie, Onni Aarne, Nora Ammann, and David Dalrymple. Flexible hardware-enabled guarantees for AI compute. *arXiv preprint arXiv:2506.03409*, 2025. URL https://arxiv.org/abs/2506.03409.

Girish Sastry, Lennart Heim, Haydn Belfield, Markus Anderljung, Miles Brundage, Julian Hazell, Cullen O'Keefe, Gillian K. Hadfield, Richard Ngo, Konstantin Pilz, George Gor, Emma Bluemke, Sarah Shoker, Janet Egan, Robert F. Trager, Shahar Avin, Adrian Weller, Yoshua Bengio, and Diane Coyle. Computing power and the governance of artificial intelligence. *arXiv preprint arXiv:2402.08797*, 2024. URL https://arxiv.org/abs/2402.08797.

Aaron Scher, David Abecassis, Peter Barnett, and Benjamin Abeyta. An international agreement to prevent the premature creation of artificial superintelligence. *arXiv preprint arXiv:2511.10783*, 2025. URL https://arxiv.org/abs/2511.10783.

Emmanouil Seferis and Tim Fist. Detecting compute structuring: Distributing training runs across providers, 2025. URL https://openreview.net/forum?id=qseqw1sWzz. Working paper.

Yonadav Shavit. What does it take to catch a chinchilla? verifying rules on large-scale neural network training via compute monitoring. *arXiv preprint arXiv:2303.11341*, 2023. URL https://arxiv.org/abs/2303.11341.

Benny J. Tang, Qiqi Chen, Matthew L. Weiss, Nathan Frey, Joseph McDonald, David Bestor, Charles Yee, William Arcand, Chansup Byun, Daniel Edelman, Matthew Hubbell, Michael Jones, Jeremy Kepner, Anna Klein, Adam Michaleas, Peter Michaleas, Lauren Milechin, Julie Mullen, Andrew

Prout, Albert Reuther, Antonio Rosa, Andrew Bowne, Lindsey McEvoy, Baolin Li, Devesh Tiwari, Vijay Gadepally, and Siddharth Samsi. The MIT supercloud workload classification challenge. *arXiv preprint arXiv:2204.05839*, 2022. URL https://arxiv.org/abs/2204.05839.

The White House. Executive order 14110: Safe, secure, and trustworthy development and use of artificial intelligence, 2023. URL https://www.federalregister.gov/documents/2023/11/01/2023-24283/. Federal Register 88 FR 75191.

Laura M. Weakley, Scott Michael, Abhinav Thota, Lee Huber, Ben Fulton, and Matthew Kusz. Classifying HPC and machine learning workloads from system telemetry. In *Proceedings of the Cray User Group (CUG)*, 2025. URL https://cug.org/proceedings/cug2023_proceedings/cug2023_doi/papers_pages/pap139.html.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

Hongyi Xu, Chen Gong, Bo Liu, Hao Zheng, Bo Chen, and Ming Li. WAVE: Workload-aware verification via performance-counter evidence for GPU inference. In *Proceedings of the ACM*, 2026. doi: 10.1145/3779212.3790247. URL https://dl.acm.org/doi/10.1145/3779212.3790247.

Jaime You and David Owen. Scaling intelligence: The exponential growth of AI's power needs. Technical report, EPRI, 2025. URL https://www.epri.com/research/products/000000003002033669.

A Related Work

The field of compute governance contains many proposals for regulations that implicitly rely on the ability to reliably identify when developers are using compute for AI training, and many proposals for hardware-enabled mechanisms that assist our classifier in securely providing that information. Shavit [2023] proposes compute monitoring for verifying rules on large-scale training. Sastry et al. [2024] provide a comprehensive framework for compute as a governance lever. Heim and Koessler [2024] analyse FLOP thresholds as a regulatory instrument; for example, the 10^26-FLOP line in U.S. EO 14110 is a direct operationalisation of this. Scher et al. [2025] propose an international agreement wherein training above certain compute thresholds is monitored and restricted or prohibited, while inference is allowed; this would require the ability to distinguish between inference and training on monitored hardware. Baker et al. [2025] identify six verification layers for international AI agreements; our work provides empirical evidence for one layer (built-in monitoring). O'Gara et al. [2025] and the RAND workshop report [Kulp et al., 2025] evaluate HEM options including workload attestation. Petrie [2025] proposes firmware-based offline licensing for export controls. Seferis and Fist [2025] argue that compute-structuring evasion (splitting training across providers) is detectable conditional on per-host workload classification; we supply and stress-test exactly that per-host classifier. Differential Architecture [Anonymous, 2026] proposes hardware mechanisms to limit performance of targeted applications, a type of enforcement complementary to our detection approach. One possibility is to robustly classify all workloads on hardware within the governance jurisdiction, and then apply the differential architecture performance restrictions to automatically enforce the established rules.

There is also existing research relevant to workload classification, which we extend. Monfared et al. [2026] study timing and memory observables for detecting training when host and device are untrusted, constructing their own derived signals from primitives that survive a strong tampering adversary. They establish signals that remain informative when NVML cannot be relied upon. Our case is complementary, establishing highly successful classification from signals that will be reliable if they can be protected from tampering by HEMs. Their work establishes a lower floor on the signal set; ours establishes an upper bound on what an adversary can hide inside that signal set. Chen et al. [2025b] develop a hardware-centric, workload-agnostic approach to hardware monitoring, analyzing over 30 ML models on various hardware platforms to perform anomaly detection using an unsupervised learning pipeline. Our scope includes supervised examples, using more than 1400 labeled records of workloads totaling over 100 hours of hardware signal telemetry to teach the classifier to recognize ML training from those signals. Gangwal et al. [2019] investigate a conceptually very similar problem,

using hardware performance counters for workload classification using RF and SVM models, but with the objective of discovering covert cryptomining, not ML training. The study notes an adversarial limitation: they predicted that evasion may be possible by restricting mining operations to obfuscate the workload signature, but at a profitability cost, analogous to the throughput penalty that evaders would suffer while attempting to evade classifiers such as ours. We extend the workload classification objective to the adversarial setting, where operators attempt to deceive the classifier.

Other past work covers several adjacent but distinct tasks. WAVE [Xu et al., 2026] identify inference workloads with 87-93% accuracy using 24 Nsight Compute per-kernel counters (1,200-5,300% overhead). We accomplish a similar-granularity classification using only always-available NVML physical counters at zero runtime overhead; the tasks are not identical (WAVE separates inference families; we separate training from inference and others), so results are not directly comparable. Esposito et al. [2025] combine NVML telemetry and performance counters to estimate GPU stress and reliability across workloads. Their results confirm that telemetry reveals meaningful workload differences, but the target is reliability estimation rather than governance-oriented training detection. ShadowScope [Almusaddar et al., 2025] uses composable performance-counter signals to detect anomalous behaviors and prevent attacks which could compromise the integrity of GPU computation. We collect and analyze similar data, but using NVML readings rather than CUDA profiling events.

Using physical side channels rather than software telemetry, Maia et al. [2022] show magnetic flux from GPU power cables reveals neural network topology. Arefin and Serwadda [2025] demonstrate USB/HDMI port power measurement can infer GPU activities. Latif et al. [2025] empirically measure power signatures of training on 8-GPU H100 nodes. These establish that physical signals carry workload-identifying information; we extend this to software-accessible telemetry for governance. POLCA [Patel et al., 2023] characterize LLM power consumption profiles across a Microsoft Azure fleet for energy planning and find systematic differences in power patterns between training and inference, providing independent evidence for the signal detected by our classifier, at the scale of large cloud datacenters.

B Classifier Design

B.1 Deployed Classifier

The operational form we deploy is a two-stage pipeline that factors the decision along a natural tree:

- **Is the workload ML?** A binary RandomForest head using all features extracted from telemetry readings classifies the workload into `ml_training` ∪ `ml_inference` vs other. The ML-vs-non-ML boundary leans heavily on level signals (peak memory footprint, sustained tensor-core utilization, power draw), so the full feature set is retained.
- **If the workload is ML, is it training?** A binary RandomForest head trained only on records where the true category is training or inference classifies incoming readings into either category. This head receives a subset of features with level features dropped (absolute memory/power/clock means, percentiles, min, max, range). The train-vs-infer decision rests on shape, periodicity, FFT, and autocorrelation terms only. This prevents the classifier from overfitting to specific models previously tested and specific power, memory, or compute throughput levels associated with particular GPUs. This also makes the classifier more robust to certain adversarial strategies: for example, the evader cannot modulate their usage levels to match that of another chip that is specifically designed for inference, because the classifier does not see the absolute levels, only the utilization ratios and temporal patterns.

The two-stage form matches the governance decision surface, makes both stages independently auditable and interpretable, and avoids the vulnerability where a single multi-class head is uncertain between training/inference, splits probability between them, and labels a non-ML workload as most probable.

B.2 Classifier Selection

We evaluated several types of models and selected the candidate that performed best in hold-out cross-validation testing. The strongest were XGBoost (n_estimators=500, max_depth=6,

lr=0.05, sample_weights via effective-number-of-samples class balancing) and RandomForest [Breiman, 2001] (n_estimators=400, max_depth=None, min_samples_leaf=2, max_features="sqrt", class_weight="balanced"), which we evaluated head-to-head on 5s to 120s windows with 5-fold cross-validation. XGBoost wins on both accuracy and fold-to-fold standard deviation at seven of nine window sizes; RandomForest leads on both at 5s and 15s. We report XGBoost at 30s as the headline operating point, but note that RF performs nearly equally well, so the main findings of this study do not depend on the particular type of classifier selected.

We also tested SVM-RBF (RBF kernel, standardised features, balanced class weights) and Logistic Regression (saga solver, L2 regularisation, standardised features).

Classifier	Accuracy	Macro-F1
XGBoost	**99.02% ± 0.71%**	**0.990**
RandomForest	97.69% ± 2.96%	0.976
SVM-RBF	96.34% ± 2.20%	0.962
Logistic Reg.	89.86% ± 1.33%	0.896

Table 6: Performance of each type of classifier at identifying held-out workloads during cross-validation.

The tree ensembles dominate by ~2 pp over SVM-RBF and ~8 pp over logistic regression, which is consistent with the feature set carrying non-linear interactions that a kernel-SVM or linear classifier cannot exploit as cheaply as a tree ensemble. SVM-RBF at 96.2% is a useful cross-check that the tree-ensemble headline is not an artifact of the specific algorithm family. The drop to 89.8% for logistic regression quantifies the non-linearity of the problem: a purely linear decision boundary in the 182-feature space is ~8 pp worse, confirming that the classifier is exploiting feature interactions (power CV × memory slope, autocorrelation × FFT periodicity, etc.) not just per-feature thresholds.

B.3 Feature Extraction

We take 5s, 15s, 30s, and 60s sliding-window snapshots of telemetry readings and extract 182 features per window. The composition is:

- Base statistics (9 signals × 13 stats, plus one additional memory statistic = 118 features): mean, std, min, max, percentiles (25/50/75/95), IQR, range, CV, skew, kurtosis.
- Autocorrelation (3 signals × 5 lags = 15 features): lags 1/2/5/10/20 for utilization, power, memory.
- Cross-signal features: power/utilization ratio, PCIe total, utilization per SM%.
- Training-specific temporal features: memory slope, epoch periodicity (via FFT of power).
- Memory-plateau features: absolute and percentage change in memory usage in first 30s, and time to memory plateau. These were motivated by an adversarial strategy that learned to spoof an earlier classifier by pre-allocating memory before it was needed for training, enabling it to hide memory allocation during training.
- Inference-specific features: duty cycle, idle fraction, utilization transitions.

B.4 Adversarial Strategy Catalog

Families are ordered by round. Baseline and Hardened columns report the binary classification accuracy (or accuracy range, for strategies with variants). The LOO column reports the accuracy (or mean accuracy, for strategies with variants) when training the classifier on evasion strategies from the same and earlier rounds, but holding that row's strategy out of the training data.

The hardest strategy for our classifier is symgemm_L4_fused_sgd, which uses a custom matrix multiplication implementation to reshape backpropagation to have the compute and memory footprint of two forward passes. All R1–R5 families are detected at 90–100% in the in-distribution hardened state.

Family	Round	Technique	Baseline	Hardened	LOO mean	Variants
A_util_modulation	1	Utilization manipulation	30%	99%	**34%**	1
B_low_util	1	Utilization manipulation	**30%**	100%	**30%**	1
D_temporal_disruption	1	Utilization manipulation	**86%**	100%	93%	1
E_memory_minimal	2	Memory reduction	**77%**	100%	100%	1
F_interleave	2	Workload mimicry	**15%**	100%	100%	2
G_clock_throttled	2	Utilization manipulation	**17%**	99%	99%	1
H_mimicry	2	Workload mimicry	**58%**	100%	100%	2
I_stochastic	2	Utilization manipulation	**46%**	96%	92%	1
J_pid	2	Utilization manipulation	**63%**	100%	100%	1
K_online_learning	3	Training-as-inference	**89%**	90%	**87%**	1
L_diluted	3	Training-as-inference	99%	100%	100%	4
M_composite_memmin	4	Composite	**10%**	100%	100%	4
N_grad_accum	4	Training-as-inference	100%	100%	100%	3
O_composite_idle_pad	4	Composite idle-padding	**64%**	100%	**85%**	11
K_ddp	4	Multi-GPU DDP	100%	100%	100%	1
K_ddp_accum	4	Multi-GPU DDP	99%	100%	100%	2
L_ddp	4	Multi-GPU DDP	98%	100%	100%	2
L_ddp_stagger	4	Multi-GPU DDP	100%	100%	100%	1
whitebox_full	5	Whitebox dilution + ckpt	97%	100%	91%	8
whitebox_lora	5	Whitebox LoRA + dilution	**50%**	92%	**56%**	3
symgemm_L4_fused_sgd	6	Matmul disguise: fused backprop	**12%**	**87%**	**12%**	6
symgemm_ppo_fused	6	Matmul disguise: diluted PPO updates	100%	100%	96%	7

Table 7: classifier's detection rates against each evasion family. Bolded baseline cells mark families with at least one variant detected at less than 90%. Bolded LOO cells mark family means detected at less than 90%.

C Power Validation

While testing 49 workloads on the RTX 5080 desktop workstation, wall-power readings were collected using a TP-Link Tapo P115 smart plug (readings logged and timestamped using the tapo python library).

The P115 smart plug collected workstation power draw readings at 1 Hz simultaneously with NVML. Across 49 runs, we collected 13,571 paired, timestamped readings (median offset 78ms, mean offset 128ms). The workloads tested include an idle baseline, 12 non-adversarial workloads (ML training, ML inference, HPC, cryptomining, rendering) and 36 adversarial runs spanning all single-GPU strategies implemented in Round 1 through Round 4.

The system idle baseline was 48W wall power while the GPU was idle: 26W system overhead + 22W for idle GPU based on NVML.

There is a strong correlation between power logged by NVML and wall power, but the slope of an unfiltered fit is inflated by adversarial strategies that run the CPU at 180–230 W (strategies B, D, K, L, N). Therefore, we report two regressions: one restricted to the 13 non-adversarial workloads ("clean", 3,698 paired seconds), and one across all 49 workloads ("all", 13,571 paired seconds).

$$\text{Clean:}\quad \text{Wall} \approx 0.97 \times \text{NVML} + 76.1\,\text{W} \qquad (r = 0.895,\ R^2 = 0.80,\ \text{RMSE} = 38\,\text{W},\ n = 3{,}698)$$

$$\text{All:}\quad \text{Wall} \approx 1.50 \times \text{NVML} + 42.7\,\text{W} \qquad (r = 0.878,\ R^2 = 0.77,\ \text{RMSE} = 62\,\text{W},\ n = 13{,}571)$$

The clean dataset's slope of 0.97 confirms that NVML accurately tracks marginal GPU power, and its intercept of 76W reflects the fixed overhead of the workstation computer while real workloads are running.

Overhead (wall power minus NVML-reported GPU power) averages 107.8 ± 64.2W for all workloads, or 68–80W for non-adversarial ML and HPC workloads, compared to 180–230W for adversarial strategies B_low_util, D, K, L, and N, which trigger heavy CPU activity while attempting to evade detection through GPU telemetry. This activity is invisible to NVML but shows up at the wall; the strategies with heavier CPU overhead are the reason that R^2 is 0.77 rather than the 0.80 observed on non-adversarial workloads.

The wall signal is richer than NVML alone, because it captures the host workstation costs incurred by strategies that manipulate GPU utilization. A layered monitoring system that combines NVML with wall or rack-level power measurements will detect evasion that NVML alone misses.

D Roofline Analysis

We establish that static architectural heuristics such as the roofline model [Williams et al., 2009] are insufficient to detect ML training. We perform a 286-configuration corner-case sweep on H100 NVL, spanning 6 workload families:

- **CNN inference** (ResNet-18/50/101/152, 95 configs): enters training band via batch-size scaling
- **LLM prefill** (GPT-2/M/L, 115 configs): FP32 prefill nearly exactly matches training arithmetic intensity
- **LLM decode** (GPT-2/M, 30 configs): contrasting case — KV-cache traffic keeps it memory-bound
- **Quantization variants** (ResNet-50/GPT-2, 18 configs): shifts arithmetic intensity without changing task semantics
- **Forward-only execution** (ResNet-50, 9 configs): closest semantic neighbor to training — preserves forward compute while removing backward/optimizer
- **ViT inference** (ViT-S/B/L, 19 configs): enters training range at unit batch

Result: 101/286 (35.3%) non-training configurations fall inside the training arithmetic-intensity range. Therefore, a temporal telemetry approach is required.

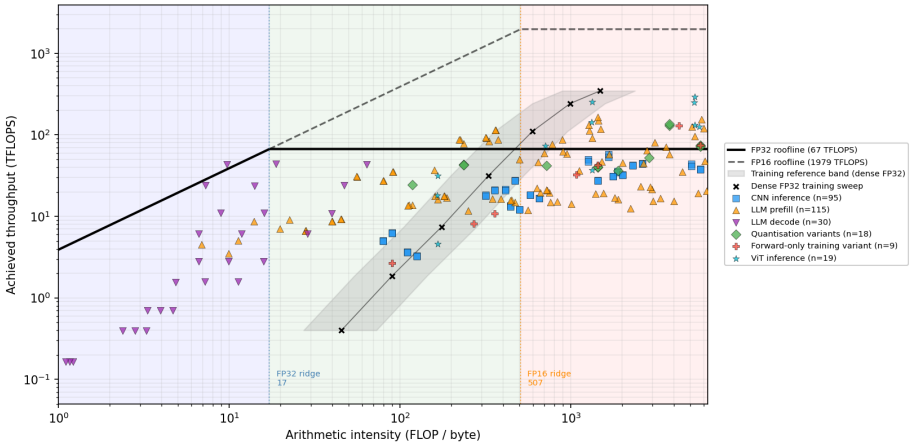


Figure 2: H100-NVL roofline with the 286 corner-case inference configurations overlaid on a dense-FP32 training reference sweep. Color-coded by workload family (CNN inference, LLM prefill, LLM decode, quantisation variants, forward-only training variant, ViT inference). The CNN-inference, LLM-prefill, and quantisation clouds land squarely inside the training AI band, confirming visually that static heuristics cannot separate these inference workloads from training.

E Window Size vs Accuracy

Classification accuracy is compared across 5/10/15/20/30/45/60/90/120 s windows. Detection is fairly flat (97.91–99.18%), so shrinking the window size to 5 s would cost barely 1% in performance. This means that a monitoring system can be configured to operate with very quick latency without significant accuracy loss.

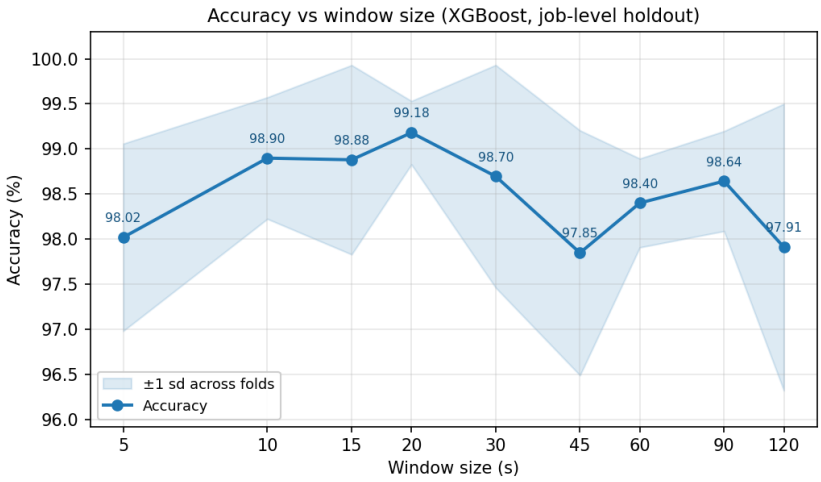


Figure 3: accuracy vs window size.

F Holdout Methodologies

We compare three different protocols for splitting the windows in the dataset into training and test sets and evaluating the classifier. The three protocols differ in how they group traces within the dataset, and in what questions they answer. Our corpus contains 1,558 runs from 1,198 jobs in 194 families.

- **Run-level** holdout groups all windows from a run together, where a run is the telemetry transcript of a GPU from the beginning to the end of its job. Jobs can run on multiple GPUs, so under run-level holdout, it is possible for data from one GPU to appear in train and another GPU to appear in test. Since traces from different GPUs running the same job are often nearly identical, this would contaminate the test set, so we do not use it.
- **Job-level** holdout groups all traces from a job together, where a job is the combined execution of a workload program on some number of GPUs (in this study, 1–8). Our corpus sometimes contains multiple jobs that are variations on the same workload, so a classifier evaluated under job-level holdout on a particular job may have seen a different variation of that job in training, but never the specific job it is being tested on. This is the main protocol reported in this study.
- **Family-level** holdout groups all jobs from a family of workloads together; for example, if one adversarial job diluted training 50:50 with inference and another job did the same training and inference but in a 20:80 ratio, both jobs are within the same family. This is the most important measurement for understanding generalization to completely novel workloads, and we report it whenever making claims about generalization.

F.1 Comparison of results by protocol

The headline result is demonstrated here, calculated using a random forest under each protocol. Each figure is the mean over five fold-assignment seeds with the forest seed held fixed; the interval is one standard deviation across those seeds.

protocol	groups	jobs split	accuracy	precision	recall	specificity
run-level	1,477	245	99.15 ± 0.15	99.33 ± 0.18	99.20 ± 0.13	99.10 ± 0.25
job-level	1,116	0	97.66 ± 0.37	97.21 ± 0.24	98.37 ± 0.98	96.22 ± 0.35
family-level	161	0	86.34 ± 1.91	82.83 ± 1.05	97.19 ± 0.55	72.95 ± 1.95

Table 8: random forest classifier performance metrics evaluated under each telemetry grouping protocol for cross-validation.

The important structure in this table is not the accuracy column. Recall is nearly invariant across the three protocols, spanning 97.19% to 99.20%, while specificity falls from 99.10% to 72.95%. The choice of protocol does not widely change whether the detector recognises training it has not seen; it changes how often the detector mistakes unfamiliar honest work for training. The eleven-point accuracy gap between run-level and family-level grouping is almost entirely driven by false positives under family-level holdout.

These false positives are concentrated rather than diffuse. Under family-level holdout, the fifteen worst benign cells account for 98.9% of all false positives, and two cells account for 54% of them. False-positive rates under job-level and family-level grouping, for the eight largest contributors:

Cell	Class	Windows	FP Job-level	FP Family-level
mining_ethash_proxy	non-ML	1,255	0.0%	81.5%
resnet50_inference	inference	1,022	0.0%	100.0%
llm_infer_deepseek_v3_tp8	inference	1,272	25.2%	39.2%
llm_infer_sdxl	inference	327	4.6%	91.4%
llm_infer_mistral_7b_q4	inference	503	3.0%	43.9%
rendering_proxy	non-ML	1,021	0.0%	17.4%
blender_bmw	non-ML	169	1.2%	76.9%
whisper_inference	inference	117	0.9%	97.4%

Table 9: false-positive rates across eight benign families highlighted for their difficulty to distinguish from training.

Aggregated by class, inference workloads go from 4.3% to 27.0% false positives and non-ML GPU work from 1.1% to 28.5%. The two failure modes are sustained non-ML compute (mining and rendering proxies, Blender) and large-model inference, which are precisely the workloads a shared research cluster contains in quantity. A deployment facing a workload mix the detector was not fitted on should expect roughly one false alarm in four on those classes.

F.2 Which protocol to use

Job-level grouping is the main-body protocol. It is the strictest partition with no measured leakage, and it matches the unit an operator actually monitors: a job, not an individual GPU stream.

Family-level grouping answers the harder question of whether the detector generalises to a workload family it has never encountered. We report it wherever a generalisation claim is made, and it is the figure a reader should use when estimating false-alarm load on an unfamiliar cluster.

Run-level grouping splits 245 jobs across its folds, so it is not used and should not be read as an estimate of deployed performance.

G Job-Level Evaluation

Results reported elsewhere in the paper concern classification of *windows* from jobs of unknown workload type, but ultimately, the governing authority needs to know the category of the whole job. Therefore, we evaluate further statistics and methods on job-level classification.

G.1 Setup and operating point

Every figure in this section comes from the same corpus and classifier as the main results: 36,666 windows from 1,558 runs across 1,198 jobs, 30 s windows at 15 s stride, XGBoost, scored out of fold under 5-fold cross-validation grouped by job, so no run is ever scored by a model that saw its own job. Results are averaged over five fold-assignment seeds and reported as mean $\pm$ standard deviation.

Window-level reference. The per-window detector reaches ROC AUC 0.9993 ± 0.0005 and average precision 0.9995 ± 0.0003, with expected calibration error 0.0046 over ten equal-width bins. At the operating threshold $\tau = 0.75$ it flags $98.14\% \pm 0.63\%$ of training windows and $0.58\% \pm 0.25\%$ of non-training windows.

Aggregation rule. We flag a run as training when 3 of any 5 consecutive windows exceed $\tau = 0.75$. The rule is causal, using only windows that have already closed, so its alert time is a quantity an

operator would actually observe. The false-alert pool is every non-training run in the corpus: 606 runs totalling 65.6 run-hours.

Rule	Detection	Missed runs	False alerts per benign run	False alerts per run-hour	Median time to first alert
3-of-5 at τ = 0.75	85.61% ± 0.44	14.39%	1.19%	0.110	60 s
CUSUM at matched false alarms	81.43% ± 1.31	18.57%	1.12%	0.104	84 s
Either rule (hybrid)	85.82% ± 0.27	14.18%	1.35%	0.125	60 s

Table 10: Run-level alert rules at a fixed operating point. A run is flagged when 3 of any 5 consecutive 30 s windows exceed τ = 0.75; CUSUM accumulates P(training) - 0.25 per window and fires at h = 2.75, chosen so its benign false-alert rate does not exceed the 3-of-5 rule's on the same runs. Detection and false alerts are means over five fold-assignment seeds; the false-alert pool is 606 non-training runs totalling 65.6 run-hours.

Short runs account for almost all of the misses. Of the 134 training runs the 3-of-5 rule fails to flag, 130 contain fewer than three windows, which is fewer than the rule needs to fire at all: a run shorter than about 60 s of telemetry cannot produce three closed windows. Conditional on a run being long enough for the rule to be able to fire, detection is 99.5% (818 of 822). By run length, detection is 0% at 1–2 windows, 96.1% at 3–4, 100% at 5–9, 99.1% at 10–19, and 100% at 20–49 and beyond. The missed runs are ordinary short fine-tuning jobs — bert_sst2, gpt2_wikitext2 and resnet_cifar10 variants account for 85 of the 134 — and only two of them belong to an evasion family. The headline 14.4% is therefore a statement about how short some of our runs are rather than about how well the detector works, and an operator monitoring jobs that last minutes rather than seconds should expect the conditional figure.

Latency. Median time to first alert is 60 s, the 95th percentile is 60 s, and the maximum is 474 s; 97.2% of flagged runs alert within 60 s and 99.8% within five minutes. The 60 s median is a structural floor rather than a measurement, since a rule requiring three closed 30 s windows at 15 s stride cannot fire sooner. This reconciles the two latency figures quoted during review: the 7 s figure described a single window crossing the threshold, which is not the rule we deploy, and 60 s is correct for the 3-of-5 rule.

G.2 Separability by evasion family

Figure 4 plots ROC and precision-recall for each evasion family against the full non-training window pool. Separability is high everywhere: ROC AUC is at least 0.9965 for every family and exactly 1.0000 for fifteen of the twenty-two. The hard families are distinguished by average precision rather than by AUC — whitebox_lora 0.9000, symgemm_L4_fused_sgd 0.9571, I_stochastic 0.9666 and K_online_learning 0.9794, against at least 0.9958 for every other family. True positive rate at a 1% false positive rate is 0.920 for symgemm_L4_fused_sgd, 0.971 for I_stochastic, 0.980 for K_online_learning, and 1.000 for the rest. The classifier therefore separates every family we constructed from benign activity, and the difficulty in the hard cases is threshold placement, not separability.

G.3 A memoryful alert rule

The 3-of-5 trigger is memoryless: evidence that never clears τ in any single window is discarded no matter how long it persists. That is exactly the wrong property for a disguise that holds P(training) consistently just below the threshold, which is what the symmetric-GEMM and white-box LoRA strategies do. We therefore also evaluate a one-sided cumulative sum: $S_0 = 0$, $S_t = \max(0,\ S_{t-1} + p_t - k)$ with reference k = 0.25, alerting when $S_t \geq h$. Inference runs drift the statistic to zero; training runs accumulate.

Calibration. We set h to the smallest value whose realised false-alert rate on benign runs is no worse than the 3-of-5 rule's own rate on the same runs, giving h = 2.75 (2.75–3.75 across seeds). Matching false alarms rather than choosing a round number is what makes the comparison meaningful: a sequential test with a free threshold can always be made to look better than a fixed one.

In distribution, CUSUM does not help. It detects 81.43% of training runs against the 3-of-5 rule's 85.61%, and its median latency is 84 s rather than 60 s. Both gaps have the same cause as the misses above — accumulating to h = 2.75 takes about four windows, so CUSUM arms more slowly than a 3-of-5 rule and fires on only 31.4% of runs that are 3–4 windows long, against 96.1% for 3-of-5. On runs of five windows or more the two rules are both at 100%.

G.4 Transfer to unseen strategies

The case the rules actually differ on is a family the detector has never trained on. We refit excluding one evasion family at a time and apply both rules at the thresholds calibrated above, changing nothing else. CUSUM never does worse on any of the twenty-two families, and improves four of them — every one of which is a family the memoryless rule struggles with.

Family (held out)	Mean window P(training)	Windows above τ	3-of-5	CUSUM	Hybrid
symgemm_L4_fused_sgd	0.107	2.7%	8.3%	25.0%	25.0%
whitebox_lora	0.504	40.0%	50.0%	83.3%	83.3%
O_composite_idle_pad	0.780	69.2%	71.9%	79.7%	81.2%
whitebox_full	0.802	66.5%	87.5%	100.0%	100.0%
B_low_util	0.661	66.8%	70.0%	70.0%	70.0%
K_online_learning	0.944	93.1%	94.4%	94.4%	94.4%
Mean over all 22 families	—	—	90.1%	93.3%	93.4%

Table 11: Transfer to unseen evasion families. The classifier is refit with one evasion family held out entirely, then both rules are applied at the thresholds calibrated in Table 10 with nothing else changed. Only the six families below 100% are listed; the remaining sixteen are flagged on every run by both rules. CUSUM is never worse than the 3-of-5 rule on any family.

The remaining sixteen families are flagged on every run by both rules. The pattern is consistent with the motivation: where the per-window evidence is weak but persistent, accumulating it recovers detections that thresholding discards. The fused-write kernel, the hardest workload in the corpus, triples from 8.3% to 25.0%, and white-box LoRA rises from 50.0% to 83.3%. Latency is the cost — median time to first alert under CUSUM on these families is 150 s for the fused-write kernel and 240 s for white-box LoRA, against the 60–90 s typical elsewhere — which is the expected trade for a rule that waits for evidence to accumulate.

There are two caveats. First, 25.0% run-level detection of an unseen fused-write kernel is better than 8.3% but it is not a deployable detection rate; the honest reading is that a memoryful rule narrows the gap rather than closing it. Second, successive windows overlap and are highly autocorrelated — we measure pairwise correlations above 0.97 at separations from 30 s to 4 h — so a long run supplies far less independent evidence than its window count suggests, and a CUSUM threshold chosen on independence assumptions would be badly miscalibrated. Calibrating h on benign runs that carry the same correlation structure is what keeps the comparison honest, and it is why we report a matched-false-alarm comparison rather than a matched-h one.

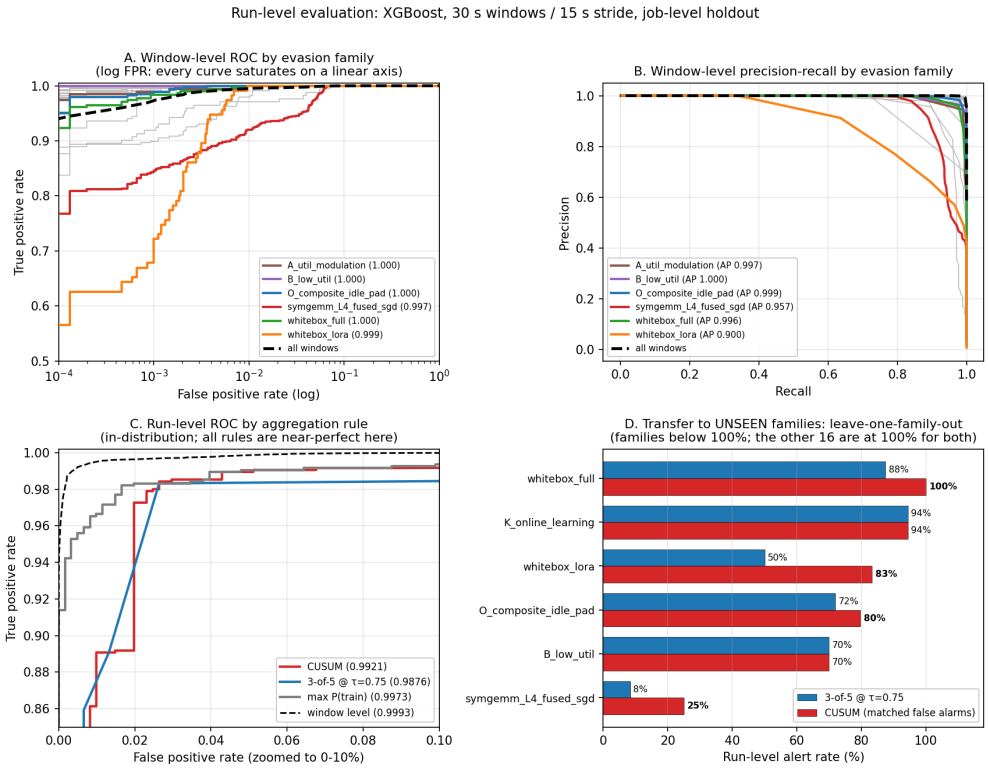


Figure 4: run-level evaluation (runlevel_roc_pr_xgb.{png,pdf}). A: window-level ROC by evasion family, on a logarithmic false-positive axis because every curve saturates on a linear one. B: window-level precision-recall by family; whitebox_lora and symgemm_L4_fused_sgd are the two visibly below the rest. C: run-level ROC for each aggregation rule, zoomed to the first 10% of false-positive rate. D: leave-one-family-out alert rates for the six families below 100%, comparing the memoryless 3-of-5 rule with CUSUM at matched false alarms; the other sixteen families are at 100% under both.

H Additional Results

Here we compare the 22 adversarial evasion strategies, sorted by detection rate in leave-one-out evaluation.

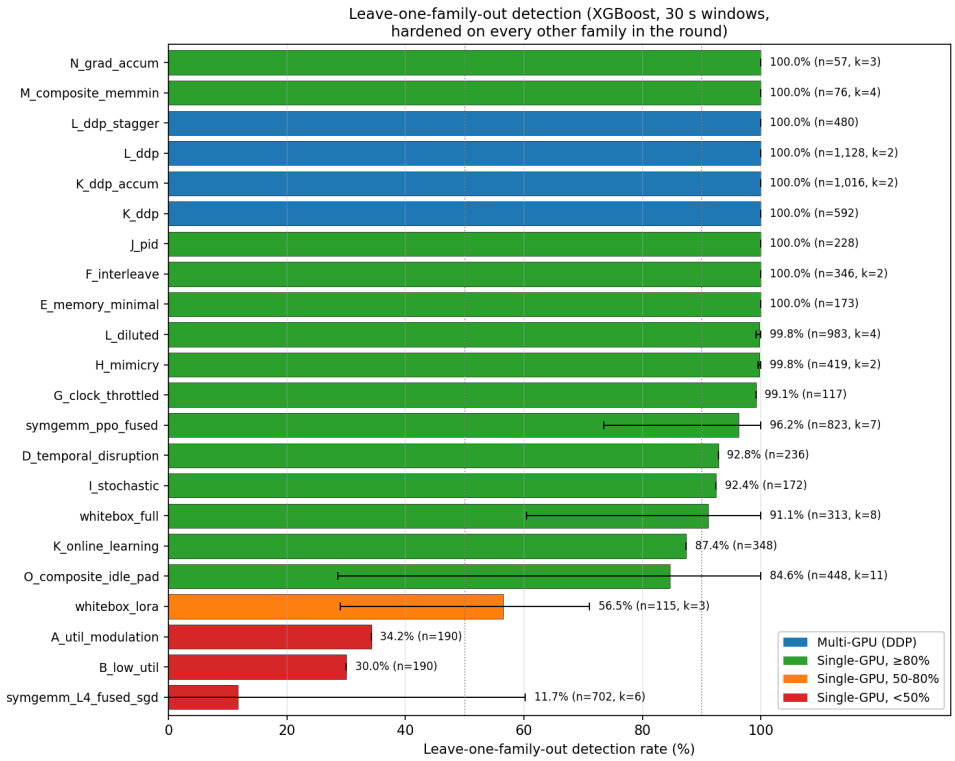


Figure 5: detection rate in LOO evaluation by strategy family. Error bars show min/max across variants within each family.

I Telemetry Visualizations

Here is a visual representation of some telemetry features transcribed from selected workloads.

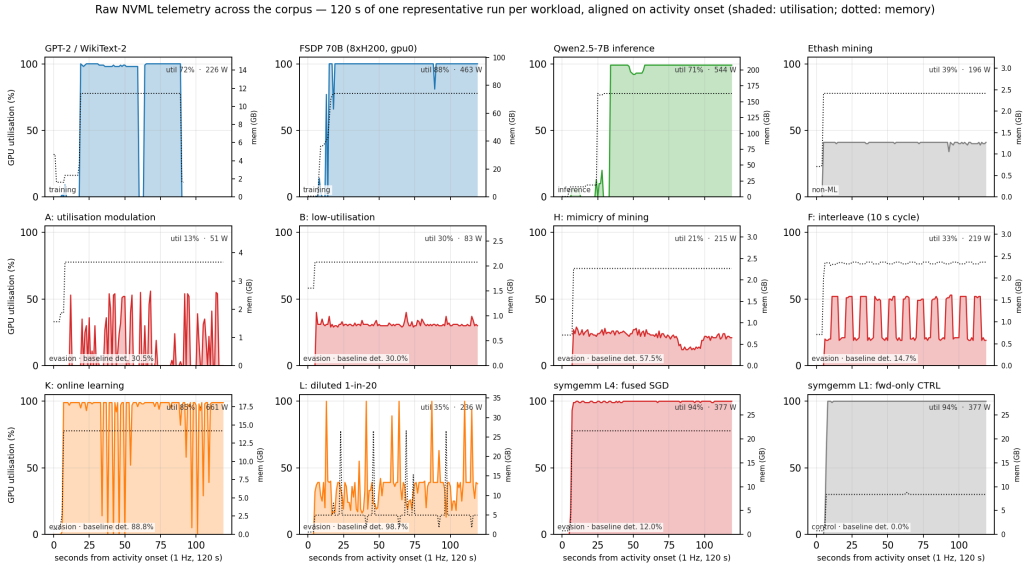


Figure 6: raw NVML telemetry for twelve workloads, 120 s per panel at 1 Hz, aligned on activity onset. Shaded area is GPU utilization (left axis), dotted line is memory in GB (right axis). Adversarial panels are labeled with their baseline detection rate from Table 7. The last two panels illustrate a matched pair: symgemm_L4_fused_sgd is disguised training, symgemm_L1_fwd3 is a similar forward-only workload, and their only difference is memory.

J Implications for HEMs

Our results naturally suggest a minimal HEM design for preventing covert training on monitored hardware. The software-level classifier demonstrates that always-available NVML counters at 1 Hz carry enough signal to detect training-versus-not at 98.7% accuracy with LOO adversarial robustness — but this result only holds if the counters themselves are trustworthy. A deployable monitoring system therefore requires three HEM components in addition to the classifier:

- **Tamper-resistant NVML read path.** The monitoring agent must read counter values from a source the adversary cannot tamper with — e.g., an attested enclave on the GPU firmware boundary [Aarne et al., 2025] or a verified read path through secure-boot-chained firmware [Petrie, 2025, O'Gara et al., 2025]. Without this, an adversary with software privilege simply returns forged values.
- **Authenticated telemetry channel.** Telemetry in transit from the GPU agent to the verifier must be signed with a hardware-rooted key so an in-path adversary cannot substitute a permitted workload's telemetry for the actual workload. This extends to rate-limiting and nonce-binding to prevent replay of previously-observed benign signatures.
- **Secure boot of the monitoring software.** The process itself, its classifier weights, and its feature-extraction code must be attested so an adversary cannot replace them with a permissive version (e.g., a classifier that labels everything as inference).

K Limitations and Future Work

This work has several limitations. First, our experiments were conducted exclusively on single-node hardware configurations, testing workloads on instances up to only 8 GPUs. Future experiments should expand to greater GPU counts, including multi-node clusters. Evasion characteristics for multi-node model- and data-parallel training will differ due to different types of interconnect (such as InfiniBand between servers vs NVLink within servers), but it is likely that training will continue to be highly distinctive due to the communication requirements of large-scale training. Relatedly, model sizes were limited by memory constraints, with largest usable models of 70B parameters for training

and 671B MoE for inference. Today's frontier models are often much larger and can require more than 8 GPUs to train and serve.

Second, our adversarial strategies were limited to framework-level evasion implemented in pytorch. An adversary with access to custom CUDA kernels could potentially manipulate individual kernel behavior, which we did not test. Future research should investigate the effectiveness of these strategies and see how robust the classifier becomes against them.

Third, the integrity of the telemetry channel is a critical assumption. Our evaluation assumes the adversary cannot tamper with NVML readings or performance counter values, nor inject synthetic telemetry to spoof a workload signature. These guarantees require a HEM for tamper-resistant telemetry and an authenticated channel. Other work like Monfared et al. [2026] investigates what can be done if these guarantees are not available.

Fourth, we only tested on NVIDIA GPUs released since 2020. This category includes the large majority of compute capacity available to cloud tenants, but substantial amounts of other chips exist, including AMD and Huawei GPUs and custom silicon such as Google TPUs and Amazon Inferentia and Trainium [Epoch AI, 2025]. Future work should generalize this approach using the analogous telemetry signals available for other hardware.

Fifth, our telemetry was sampled at ~1 Hz, a frequency which cannot detect very brief events. Future work should test other sampling rates, measuring how much classification improves with more frequent sampling, how much sampling can be done without degrading performance, and investigating the minimum sample rate that is still difficult to evade.

Finally, we only tested a limited range of workloads and adversarial strategies. Real datacenters run a wider variety of applications than we emulated, with notable gaps including multi-user inference serving, data processing and ETL pipelines, and predictive analytics. Although we tried to cover many evasion strategies and edge cases, real evaders might attempt a much wider range of tactics that we cannot anticipate. It is important to continue iterating on this work to make the classifier even more robust to unknown future threats. Iteration should include more white-box rounds, to determine whether resistance to white-box attacks improves generally, or if an evader can always defeat a classifier if they obtain white-box access, and if so, how much performance penalty is incurred in the process.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the 98.7% binary accuracy headline, the six-round adversary–defender protocol over 22 evasion families and 9 GPU models, and the 12–91% range against the hardest unseen disguised workloads. Section 4 and Appendix K report the cases where detection degrades.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Appendix K discusses single-node scope ($\leq$8 GPUs), framework-level-only evasion, the tamper-resistant-telemetry assumption, NVIDIA-only hardware, 1 Hz sampling, and limited workload coverage. Section 4.1 reports substantially weaker leave-one-GPU-out accuracy on consumer hardware (67–80%), and Appendix F reports the false-positive cost of family-level holdout.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper is empirical. It contains no theorems, formal claims, or proofs.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 3 specifies the nine NVML counters, the 1 Hz sampling rate, the window sizes, and the feature extraction; Appendix B gives the classifier architecture and hyperparameters; Appendix F specifies the three grouping protocols and which one is reported. Source code for every workload is available in the project repository.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Source code for the non-ML, inference, training, and adversarial evasion workloads is available in the project repository, as stated in Section 3 and the Conclusion.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Appendix B.2 reports the candidate models and their hyperparameters (XGBoost: 500 trees, max_depth 6, learning rate 0.05, effective-number-of-samples class balancing; RandomForest: 400 trees, min_samples_leaf 2, max_features sqrt, balanced class weights). Appendix B.3 enumerates the 182 features. Appendix F defines the run-, job-, and family-level splits and states that job-level grouping is the protocol reported in the main body.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: Accuracies are reported as mean ± one standard deviation over five fold-assignment seeds with the model seed held fixed, so the interval isolates fold-assignment variance rather than conflating it with model variance. Table 6 reports per-classifier standard deviations, Table 8 reports them per holdout protocol, and Table 10 reports them for the run-level alert rules.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section 3 reports 1,566 runs of 194 distinct workloads totalling approximately 164 GPU-hours (87 wall-clock hours) and 601,112 telemetry readings. Table 1 lists the nine GPU models, their memory, and the node sizes (1–8 GPUs) used. Classifier fitting is inexpensive relative to data collection.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research conforms to the NeurIPS Code of Ethics. It involves no human subjects and no personal data; all telemetry was collected from the authors' own experimental workloads on hardware they operated or rented.

   Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 5.3 and the Conclusion discuss the governance benefits and the content-agnostic, privacy-preserving nature of the signals. The paper also publishes a catalogue of evasion strategies; this is deliberate disclosure intended to let detectors be hardened against them, and Appendix B.4 reports that the strategies which evade detection do so at a throughput cost.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [Yes]

    Justification: No pretrained model or scraped dataset with high misuse risk is released. The evasion strategies are released deliberately so the community can red-team and strengthen the detector, as stated in the Introduction and Conclusion; the strategies impose throughput penalties that limit their attractiveness at frontier scale, and Appendix J specifies the hardware guarantees a deployment would need.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [No]

    Justification: Existing models, datasets, and tools used as workloads (e.g. ResNet, GPT-2, BERT, TinyLlama, Qwen-2.5, Mistral, DeepSeek, Whisper, SDXL, CIFAR-10, WikiText, Blender, FFmpeg) are named in Section 3 and cited where a paper exists, but the paper does not enumerate their individual licences and terms of use.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The new assets are the telemetry corpus and the workload implementations. Section 3 documents the corpus composition and collection procedure, Appendix B.4 documents every adversarial strategy family and its variants, and the workload source code is available in the project repository.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

Justification: The research does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The research does not involve human subjects, so no IRB review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: Large language models appear in this work only as the workloads being executed and measured, not as a component of the research method. LLM assistance in writing and coding does not require declaration under the guidelines.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.